

DeployGuard

Industrial Visit Project Report

Final Year Project II - Implementation

Project	DeployGuard - Deterministic PaaS Deployment Control Plane
Group Members	Hasan Sajjad Faria Fatima Tania Khawar
Department	Department of Computer Science, Air University Islamabad
Implementation Baseline	GitHub branch test2 - reviewed 13 September 2026

Prepared as an implementation-focused FYP-II technical report.

Executive Summary

DeployGuard is a Platform-as-a-Service implementation that turns a connected GitHub repository into a repeatable, policy-controlled cloud deployment. Its central contribution is a deterministic deployment control plane: it resolves exactly what source and service should be built, creates immutable build and runtime specifications, coordinates container image publication, provisions AWS infrastructure through Infrastructure as Code, verifies the runtime, and records a stable release only after evidence-backed checks succeed.

The FYP-II implementation focuses on reliability across the complete lifecycle rather than on a single deployment command. The platform includes repository authorization, source-SHA pinning, service discovery, canonical build targets, automatic service-port resolution, environment and secret ownership, container vulnerability scanning, GitHub Actions orchestration, AWS OIDC authentication, Amazon ECR, Amazon ECS on Fargate, Application Load Balancers, CloudWatch, Secrets Manager, EFS-backed database persistence, managed PostgreSQL/MySQL/MongoDB support, deployment generations, redeploy, rollback, destroy, state reconciliation, monitoring, FinOps, billing, audit logs, and evidence-driven AI troubleshooting.

The architecture follows platform-engineering principles in which developers use a self-service interface while the platform hides low-level cloud mechanics behind governed workflows. Recent research similarly identifies internal developer platforms, Infrastructure as Code, embedded security, observability, and policy automation as key mechanisms for reducing cognitive load and improving delivery consistency [1]-[5]. DeployGuard applies these concepts to a student-built but production-oriented deployment system, with particular emphasis on deterministic contracts, explicit ownership boundaries, auditable evidence, and lifecycle correctness.

Contents

1. Introduction	16. Destroy and Cloud Cleanup
2. Problem Statement	17. Failure Diagnostics and Recovery
3. Project Objectives	18. Observability
4. Scope and Supported Application Contract	19. AI-Assisted Troubleshooting
5. Literature and Engineering Foundation	20. FinOps, Billing and Cost Governance
6. System Architecture	21. Frontend and Developer Experience
7. Core Deployment Pipeline	22. Testing and Certification Strategy
8. Canonical Contracts and Determinism	23. FYP-II Implementation Achievements
9. Service Discovery and Port Resolution	24. Demonstration Plan
10. Technology Stack	25. Limitations
11. AWS Runtime Architecture	26. Future Work
12. Infrastructure Ownership and State	27. Conclusion
13. Managed Databases	References
14. Security Architecture	Appendix A - FYP-II Defense Positioning
15. Release Generations, Redeploy and Rollback	

1. Introduction

Modern web application deployment spans source control, dependency resolution, containerization, CI/CD, cloud identity, infrastructure provisioning, runtime orchestration, secrets, databases, monitoring, security and cost. Although each layer has mature tooling, developers still face a coordination problem: a repository must be interpreted consistently, the correct service must be built, cloud resources must match the intended runtime, and every release must preserve enough evidence to support rollback and troubleshooting.

DeployGuard addresses this coordination problem as a PaaS-style control plane. The implementation does not attempt to replace cloud services or Infrastructure as Code. Instead, it owns the deterministic decisions that connect application source to a verified release. This distinction is important for FYP-II: the project contribution is the orchestration, contracts, lifecycle model and operator experience built around standard infrastructure primitives.

1.1 Project Team

Member	Role in FYP-II
Hasan Sajjad	Deployment architecture, cloud lifecycle, reliability certification and integration.
Faria Fatima	Application features, testing support, documentation and presentation preparation.
Tania Khawar	Application features, validation support, documentation and project demonstration.

1.2 Implementation Baseline

This report was regenerated against the DeployGuard GitHub test2 branch. The branch head reviewed on 13 September 2026 was commit 12ed47797fe82bee1bdec449936e57d7c27758e4. The report describes the implemented system at architectural and product level and intentionally avoids treating implementation dependencies as the project contribution.

2. Problem Statement

A developer can often deploy a simple application manually, but reliable automation becomes difficult when repositories vary in structure, frameworks expect different ports, monorepos contain multiple packages, cloud credentials must be scoped, infrastructure state must remain consistent, databases require persistence, and a failed deployment must be explained without exposing secrets.

The core problem is therefore not merely “how to run a container.” It is how to convert an arbitrary supported repository into a bounded, reproducible deployment contract and then preserve correctness through deploy, redeploy, rollback and destroy. Manual workflows create opportunities for configuration drift, stale state, inconsistent environments, hidden permission gaps, fragile recovery steps and operator uncertainty. Infrastructure-as-Code research identifies exactly these kinds of drift and reproducibility issues as reasons to formalize cloud state in code [3].

2.1 Research Questions

1. How can a PaaS resolve a reproducible build target from heterogeneous JavaScript and Python repositories?
2. How can source, build configuration, runtime configuration and release identity remain correlated across an asynchronous cloud deployment?
3. How can Infrastructure as Code and runtime orchestration share responsibility without creating conflicting authorities?
4. How can security, cost, observability and troubleshooting be embedded into the deployment lifecycle rather than added after deployment?
5. How can redeploy, rollback and destroy remain deterministic and auditable after multiple release generations?

3. Project Objectives

Objective	Implementation outcome
Self-service deployment	Users connect a GitHub repository, configure a project and initiate deployment from the platform UI.
Deterministic repository analysis	DeployGuard pins an exact source SHA and resolves a canonical service/build target before cloud mutation.
Automatic runtime configuration	The platform resolves service ports, reserved variables, secret references and managed database bindings.
Containerized delivery	A build worker creates validated immutable OCI images and publishes them to Amazon ECR.
Declarative infrastructure	Terraform provisions and destroys project-scoped AWS resources with remote state.
Managed runtime	Amazon ECS/Fargate runs application services

	behind an Application Load Balancer.
Security integration	OIDC, role-based authorization, secret isolation, vulnerability scanning and audit records are integrated into the lifecycle.
Lifecycle safety	Deployment generations, stable release records, rollback targets and destroy verification preserve release history.
Operational visibility	CloudWatch, platform metrics, Prometheus/Grafana integrations and structured failure evidence support diagnosis.
Cost awareness	FinOps estimates, thresholds, approval rules and billing/entitlement logic are exposed in the product.
AI-assisted troubleshooting	Sanitized evidence is converted into bounded diagnostic summaries and remediation guidance.

4. Scope and Supported Application Contract

DeployGuard currently targets web applications that can be resolved into supported JavaScript or Python deployment contracts. The system supports standalone JavaScript services, JavaScript workspace members, and standalone Python services. Applications may be APIs, server-rendered web applications, full-stack services or framework-served static applications. Pure static-file repositories without an application runtime are outside the current contract.

The resolver intentionally rejects ambiguous or unsupported cases instead of guessing. This fail-closed behavior is a reliability feature: an explicit unsupported result is safer than silently building the wrong directory or starting an incorrect command.

Contract	Examples / characteristics	Status
JavaScript standalone	Express, NestJS, Next.js, Nuxt and similar package-based web services.	Supported
JavaScript workspace member	Monorepo package selected by canonical workspace	Supported

	ownership and package identity.	
Python standalone	Flask, FastAPI, Django and similar Python web services.	Supported
Pure static repository	HTML/CSS/JS with no supported application runtime.	Outside current deployment contract
Arbitrary native/desktop workload	Non-web executables or workloads without a supported service contract.	Outside current scope

5. Literature and Engineering Foundation

Platform engineering research frames internal developer platforms as self-service layers that reduce infrastructure cognitive load while standardizing delivery workflows. A 2026 multivocal review found that common platform components include CI/CD, Infrastructure as Code, observability, policy enforcement and developer portals, while recommending that platforms be treated as products rather than collections of tools [1]. A systematic review similarly emphasizes golden paths, developer experience and a balance between standardization and flexibility [2].

Infrastructure as Code literature identifies reproducibility, auditability and drift reduction as major advantages of declarative cloud definitions [3]. This directly motivates DeployGuard's decision to keep Terraform as the infrastructure/state/destroy authority while the control plane owns higher-level deployment intent, runtime verification and release promotion.

DevSecOps research supports integrating automated security checks into CI/CD rather than treating security as a separate late stage [4]. Container-image vulnerability assessment research also treats image scanning as a fundamental cloud-native security control [5]. DeployGuard therefore includes a container-security gate before a release is accepted.

Research on autonomous code-to-cloud deployment demonstrates the feasibility of combining repository analysis, infrastructure reasoning and deployment automation, while also warning that fully generated infrastructure can over-provision compared with expert baselines [6]. DeployGuard takes a more bounded approach: deterministic rules and explicit contracts remain authoritative, and AI is used for troubleshooting rather than for unconstrained infrastructure mutation. A systematic review of AI for Infrastructure as Code likewise finds strong opportunities in analysis and lifecycle assistance but notes that operational phases remain less studied [7].

Ref.	Research evidence	Relevance to DeployGuard
[1]	Platform engineering +	Self-service control plane,

	internal developer portals	golden paths, governance, observability.
[2]	Platform engineering + developer experience systematic review	Abstraction, reduced cognitive load, platform-as-product thinking.
[3]	Terraform/IaC for scalable cloud deployments	Reproducibility, auditability, drift reduction, declarative lifecycle.
[4]	Automated security scanning in DevSecOps CI/CD	Security checks embedded in delivery pipelines.
[5]	Container image vulnerability assessment	Image security as a pre-release control.
[6]	Autonomous code-to-infrastructure deployment	Motivation for source analysis and integrated deployment automation.
[7]	AI for IaC systematic review	Evidence-driven AI assistance without surrendering deterministic control.

6. System Architecture

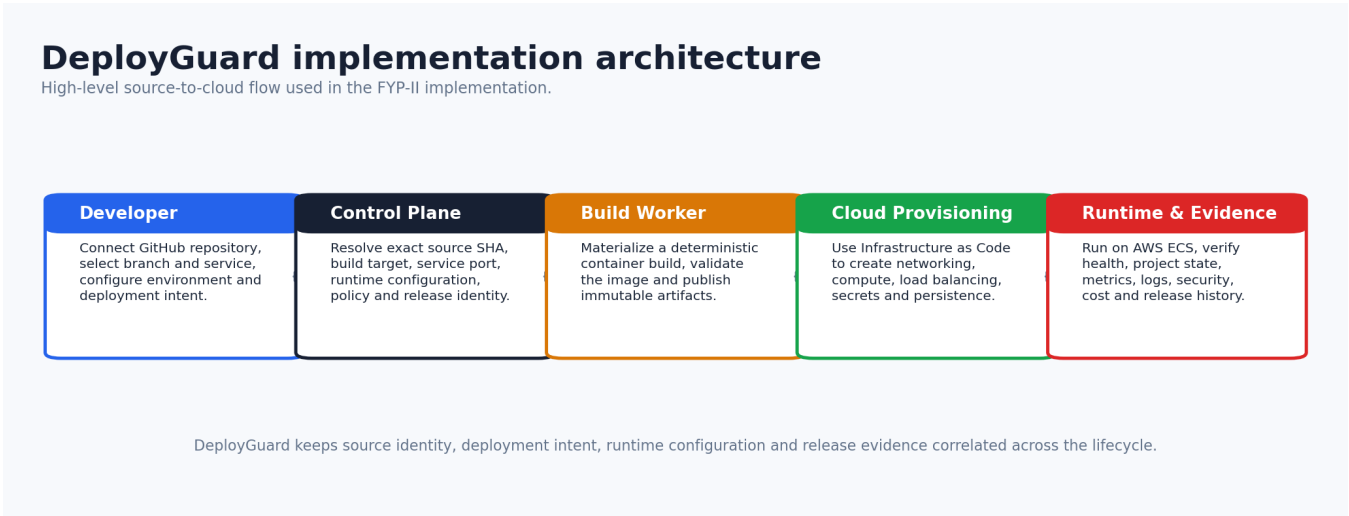


Figure 1. DeployGuard source-to-cloud architecture.

6.1 Architectural Layers

Layer	Primary responsibilities
Frontend / Developer Experience	Authentication, GitHub connection, project creation, service/environment configuration, deployment views, monitoring, cost, troubleshooting and admin surfaces.
Control Plane	Source identity, service discovery, build target, service port, configuration ownership, deployment intent, lifecycle state, failure classification and release history.
Execution Pipeline	GitHub Actions workflow dispatch, container build, validation, vulnerability scan, image publication, infrastructure execution and release evidence.
Infrastructure Layer	Terraform-owned AWS networking, ECS resources, load balancing, IAM execution roles, persistent storage, secrets and project state.
Runtime Layer	ECS/Fargate tasks, health checks, application traffic, managed database runtime, logs and metrics.
Operational Intelligence	CloudWatch evidence, Prometheus/Grafana surfaces, FinOps estimation, AI troubleshooting, audit logs and notifications.

7. Core Deployment Pipeline

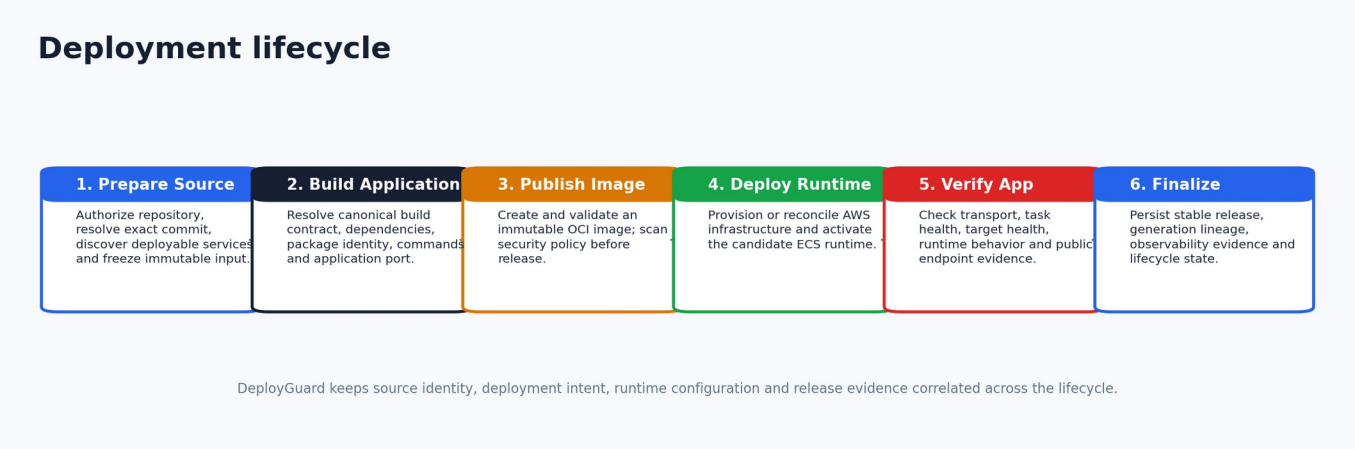


Figure 2. Six-stage deployment lifecycle used for product presentation.

7.1 Stage 1 - Prepare Source

DeployGuard authorizes the repository, resolves the exact Git commit SHA, checks project configuration, identifies deployable services and freezes the source identity used by the operation. This prevents a moving branch head from silently changing what is deployed while the workflow is running.

7.2 Stage 2 - Build Application

Repository analysis produces an immutable build contract rather than a free-form set of guesses. The contract records the selected service directory, workspace/build/install roots, package identity, execution metadata, dependency paths, resolution strategy, evidence and a SHA-256 fingerprint. Supported contracts are JavaScript standalone, JavaScript workspace member and Python standalone.

The build worker consumes this normalized specification and materializes a container image. Build-time behavior is separated from runtime configuration so secrets and runtime-only values are not unintentionally treated as build inputs.

7.3 Stage 3 - Publish Image

The produced container is validated and published as an immutable image artifact in Amazon ECR. Release identity is correlated with the source SHA, operation identity and service identity. Security scanning can operate in reporting or enforcing mode before the release advances.

7.4 Stage 4 - Deploy Runtime

Terraform reconciles project infrastructure and ECS runtime resources. DeployGuard verifies required AWS capabilities before dispatch, then tracks the operation through cloud state. Infrastructure and release ownership are separated so subsequent topology reconciliation does not unintentionally overwrite the active verified release.

7.5 Stage 5 - Verify Application

Verification combines transport readiness, ECS task/service evidence, load-balancer target health and public application checks. Managed database prerequisites are evaluated before a database-dependent service is promoted. A candidate only becomes stable after verification evidence satisfies the release contract.

7.6 Stage 6 - Finalize Release

The platform persists stable release identity, deployment generation state, runtime configuration revision, image provenance and evidence required for observability, rollback and audit.

8. Canonical Contracts and Determinism

A key implementation decision is to make deployment decisions explicit data. Instead of repeatedly re-detecting a repository at different stages, DeployGuard creates versioned immutable specifications and fingerprints. This reduces the risk of stale or contradictory interpretation between analysis, build, infrastructure and runtime stages.

Canonical object	Purpose	Selected fields
Build Target	Defines what part of the repository is buildable.	source SHA, service directory, workspace root, build root, install root, package identity, contract, execution, dependency paths, evidence, fingerprint
Runtime Configuration Revision	Defines runtime environment of a service.	service port, environment variables, secret references, database attachment, immutable revision identity
Deployment Generation	Represents one candidate/live cloud generation.	project, environment, ordinal, status, state key, resource manifest, operation lineage
Stable Release	Represents the currently accepted verified release.	generation, commit SHA, health path, deployment operation, deployed time, status
Failure Diagnostic	Normalizes terminal failure evidence.	stage, failure code, service, ownership, external provider, root cause, retry decision

9. Service Discovery and Port Resolution

Port resolution is treated as a deterministic admission problem. DeployGuard evaluates evidence in priority order: explicit application configuration, source-code binding, start-command flags, environment declarations and recognized framework defaults. Conflicting evidence fails instead of selecting an arbitrary value.

Recognized framework defaults include common JavaScript and Python frameworks, while explicit code/configuration remains higher priority. The resolved port becomes platform-owned runtime configuration; the application is expected to bind on the provided port and on an externally reachable interface.

Evidence priority	Example
1 - explicit configuration	server.port, Vite/Nuxt server configuration, package configuration
2 - source binding	server.listen(...), Flask app.run(port=...), uvicorn.run(...), Go ListenAndServe
3 - start command	--port or -p flags in start commands / Procfile
4 - environment	PORT or framework-specific port variable
5 - framework default	Next/Nest/Nuxt 3000, Vite 5173, Django/FastAPI 8000, Flask 5000

10. Technology Stack

Area	Technology / service	Purpose
Frontend	React 18, React Router, Vite	Developer and admin web interface.
Backend	NestJS 11, TypeScript, TypeORM	Control-plane API, business logic and persistence.
Control-plane database	PostgreSQL	Projects, revisions, pipeline runs, releases, audit, billing and operational state.

Async processing	BullMQ + Redis	Pipeline and lifecycle jobs.
Source / CI/CD	GitHub + GitHub Actions	Repository integration and workflow execution.
Container registry	Amazon ECR	Immutable OCI image storage.
Runtime	Amazon ECS on AWS Fargate	Containerized application and database tasks.
Infrastructure as Code	Terraform	Project infrastructure, state and destroy.
Traffic	Application Load Balancer	Public HTTP routing and target-health integration.
Secrets	AWS Secrets Manager	Runtime/application/database secret references.
Persistence	Amazon EFS	Persistent managed-database storage.
Monitoring	CloudWatch, Prometheus, Grafana	Logs, metrics, runtime and platform visibility.
Security	Trivy, IAM/OIDC, RBAC, Helmet	Image scanning, cloud identity and application security controls.
FinOps / billing	Infracost integration, Stripe modules	Cost estimation, policy, subscriptions and billing.

11. AWS Runtime Architecture

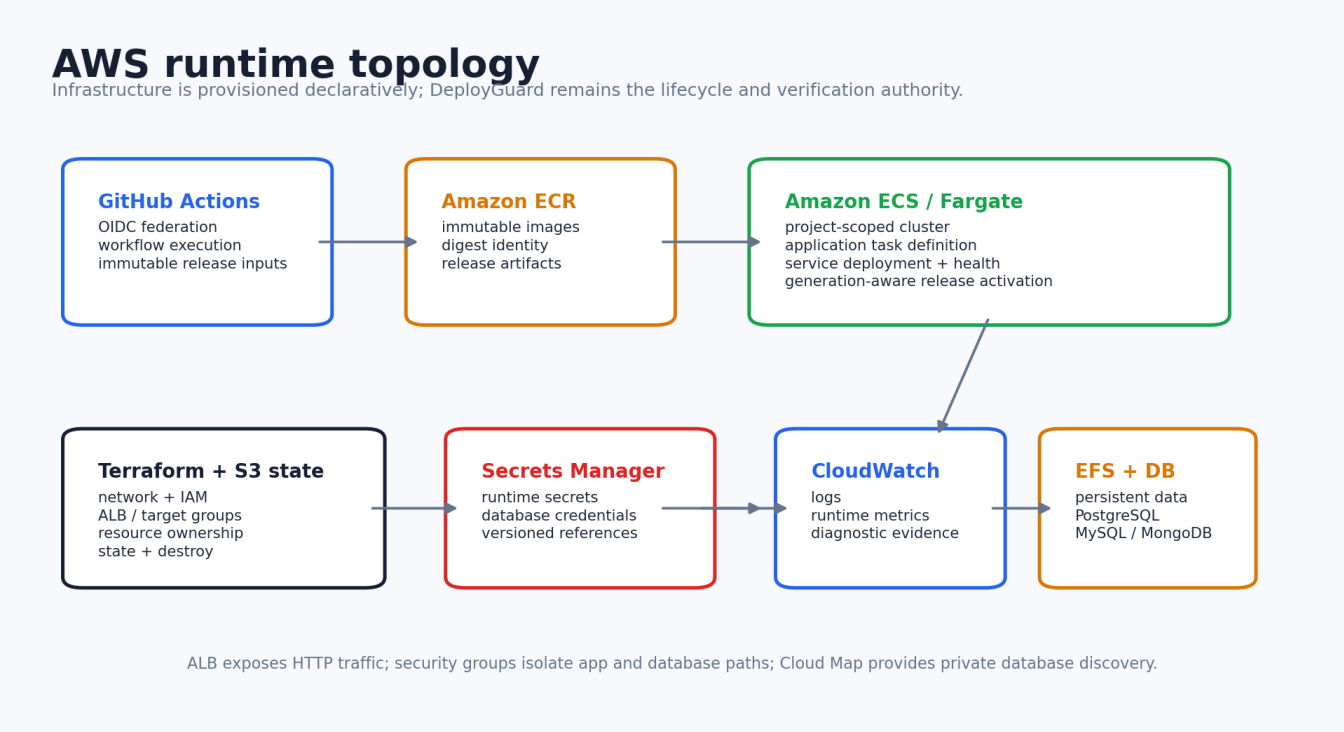


Figure 3. Project-scoped AWS runtime topology.

AWS component	Role in DeployGuard
STS / IAM	Federated workflow identity, execution roles and capability verification.
ECR	Project-scoped image repository and immutable image publication.
ECS / Fargate	Project cluster, application services/tasks and managed database tasks.
EC2 networking	VPC/subnet discovery and security groups.
Application Load Balancer	Public listener, target group and health-based traffic routing.
CloudWatch Logs	Application, database and platform diagnostic logs.
Secrets Manager	Application secrets and managed database credentials.

EFS	Encrypted persistent managed-database storage.
Cloud Map / Route 53 private namespace	Private service discovery for managed databases.
S3	Remote Terraform state.

12. Infrastructure Ownership and State

DeployGuard uses a split-authority model. Terraform is the source of truth for infrastructure resource existence and destruction. DeployGuard is the authority for application runtime acceptance, release promotion and lifecycle decisions. ECS provides low-level rolling deployment primitives.

This separation prevents two control systems from fighting. For example, once the platform updates a service to a verified release revision, later infrastructure reconciliation should preserve that active release rather than reverting it to a bootstrap task definition. Destroy remains declarative through Terraform so resource deletion follows the same state model used for provisioning.

Authority	Owns
Terraform	ECS resource existence, ALB/listener/target group, security groups, IAM execution role, EFS, service discovery, secret infrastructure, remote state and final destroy.
DeployGuard control plane	Source/build/runtime contracts, database readiness decision, runtime success/failure, diagnostics, public verification, release promotion, rollback target and project lifecycle state.
ECS	Task scheduling, service updates, deployment stability and low-level runtime primitives.

13. Managed Databases

DeployGuard can attach one managed database to a service and currently models PostgreSQL 16, MySQL 8 and MongoDB 8. Each engine has a canonical port, data path, health check, initialization

variables and URL scheme. Credentials are generated and stored in AWS Secrets Manager, while persistent data is mounted on encrypted Amazon EFS.

The database is isolated by security-group rules so inbound database traffic is accepted from the attached application security group rather than from the public internet. Private discovery names provide a stable application-facing host identity. The platform also validates database URL scheme evidence so a consumer configuration cannot silently mismatch the selected engine.

Engine	Port	Persistent path	Readiness mechanism
PostgreSQL	5432	/var/lib/postgresql/data	pg_isready + runtime health evidence
MySQL	3306	/var/lib/mysql	mysqladmin health plus grant reconciliation
MongoDB	27017	/data/db	mongosh ping with authenticated initialization

14. Security Architecture

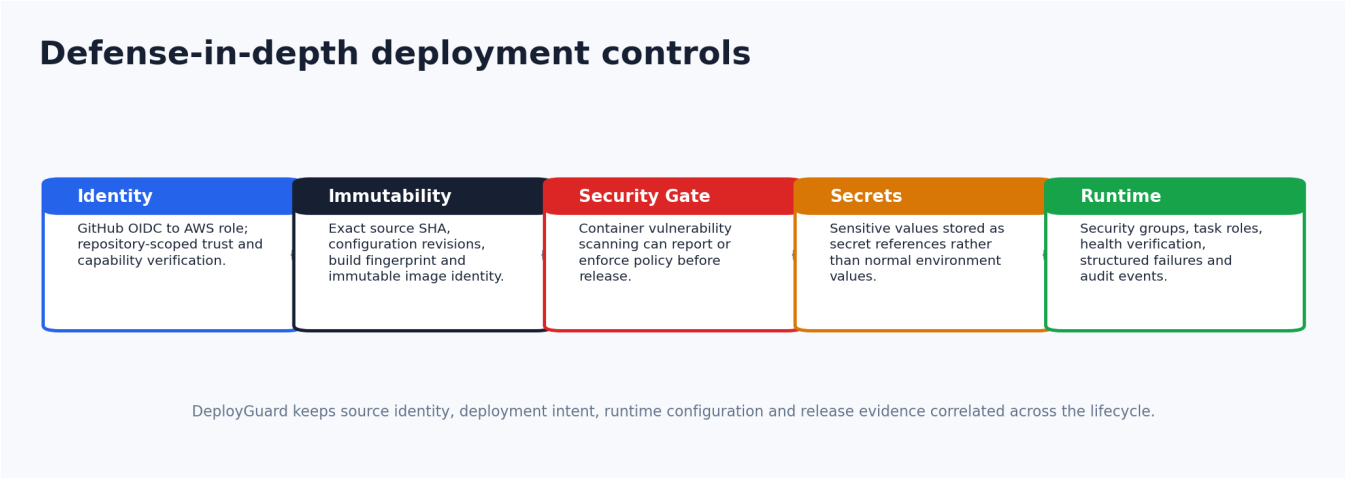


Figure 4. Defense-in-depth controls across the deployment path.

14.1 Cloud Authentication with OIDC

GitHub Actions authenticates to AWS through OpenID Connect and an IAM role trust policy rather than requiring long-lived AWS access keys in every repository. DeployGuard verifies that the configured trust policy contains the GitHub OIDC provider, the AWS STS audience and an authorized repository subject pattern.

14.2 Capability Verification

Before dispatching cloud lifecycle work, the platform maintains an explicit AWS capability model covering STS, ECR, ECS, EC2, load balancing, CloudWatch Logs, IAM, Secrets Manager, EFS, S3, service discovery and Route 53. The contract differs by deploy, rollback and destroy path and can conditionally include managed-database requirements.

14.3 Container Security

Container vulnerability scanning is controlled by platform configuration. The scanner can be enabled in non-blocking mode for visibility or enforcement mode to block a release that violates the security policy. This aligns with DevSecOps literature that recommends embedding security controls directly into CI/CD [4][5].

14.4 Secret and Environment Ownership

DeployGuard separates normal environment values from secret references and reserves platform-owned keys such as PORT/HOST and managed-database aliases. Runtime task definitions consume sensitive data from Secrets Manager rather than embedding plaintext values into source or normal configuration records.

14.5 Application and Operator Controls

The backend includes authentication, role guards, admin segregation, audit logs, request validation and security middleware. Operator surfaces for destructive lifecycle actions require explicit confirmation and are separated from ordinary developer operations.

15. Release Generations, Redeploy and Rollback

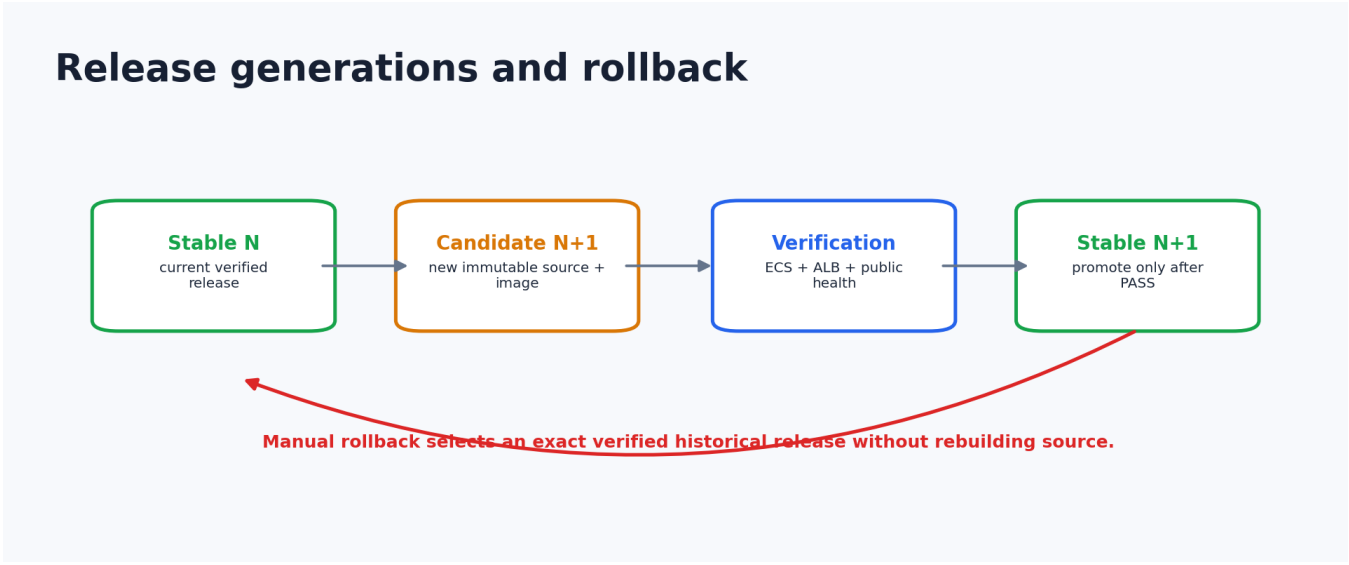


Figure 5. Candidate generation, verification, promotion and rollback.

15.1 Deployment Generations

Each cloud deployment can create a generation with states such as deploying, live, failed, retired, cleanup pending and cleaned. Database constraints enforce at most one live generation and one

deploying candidate for a project/environment. This provides explicit lifecycle lineage instead of treating the current cloud resources as the only source of history.

15.2 Redeploy

For release-only application changes, the platform can register a new immutable task definition, update the ECS service, wait for stability, verify the application and then promote the new release. This avoids rebuilding unrelated infrastructure for every code change.

15.3 Manual Rollback

Rollback selects an exact historical release identity. The platform validates the rollback target, updates ECS to the previous immutable runtime revision, waits for runtime stability, verifies health and only then moves the stable pointer. The rollback path does not rebuild the old source, which preserves reproducibility and reduces recovery time.

16. Destroy and Cloud Cleanup

Destroy is treated as a first-class lifecycle operation rather than an ad hoc delete button. The flow uses admission/confirmation, Terraform destroy, cloud deletion verification, non-Terraform cleanup and control-plane finalization. Project resource inventory and cleanup services help detect residue or orphaned cloud resources.

The implementation also contains central/admin cleanup capabilities, TTL-based cleanup support and project cloud-state reconciliation. These controls address the operational reality that failed or interrupted lifecycle operations can leave resources behind even when the normal path is declarative.

17. Failure Diagnostics and Recovery

A serious deployment platform must distinguish application failures from platform failures and external provider failures. DeployGuard stores structured pipeline runs and terminal diagnostic metadata instead of relying only on unstructured logs. Failure classification records stage, deterministic code, affected service, likely ownership, external provider and retry/recovery information.

Recovery logic is stage-aware and evidence-driven. The system can present a retry when it is safe, recommend application repair for repository-owned failures, identify external provider boundaries, or keep the state unverified when evidence is insufficient. This avoids false confidence from guessed root causes.

Classification	Meaning
DEPLOYGUARD_PLATFORM	A platform-owned contract, state, orchestration or configuration defect is supported by evidence.

REPOSITORY_APPLICATION	The repository/application is the proven source of the failure.
EXTERNAL_PROVIDER	AWS/GitHub or another external boundary is the supported failure owner.
UNVERIFIED	Available evidence is insufficient for a defensible ownership decision.

18. Observability

Observability surface	Implementation role
CloudWatch application logs	Per-service runtime logs with project/service correlation.
CloudWatch metrics	Runtime health and AWS operational metrics.
Pipeline metrics	Stage durations, pipeline outcomes and workflow execution metrics.
Prometheus endpoint	Platform metrics export.
Grafana dashboards	Visual runtime and platform monitoring.
Developer projection	Simplified project status/health view derived from canonical state.
Audit log	Actor/action/resource history for security and operations.

19. AI-Assisted Troubleshooting

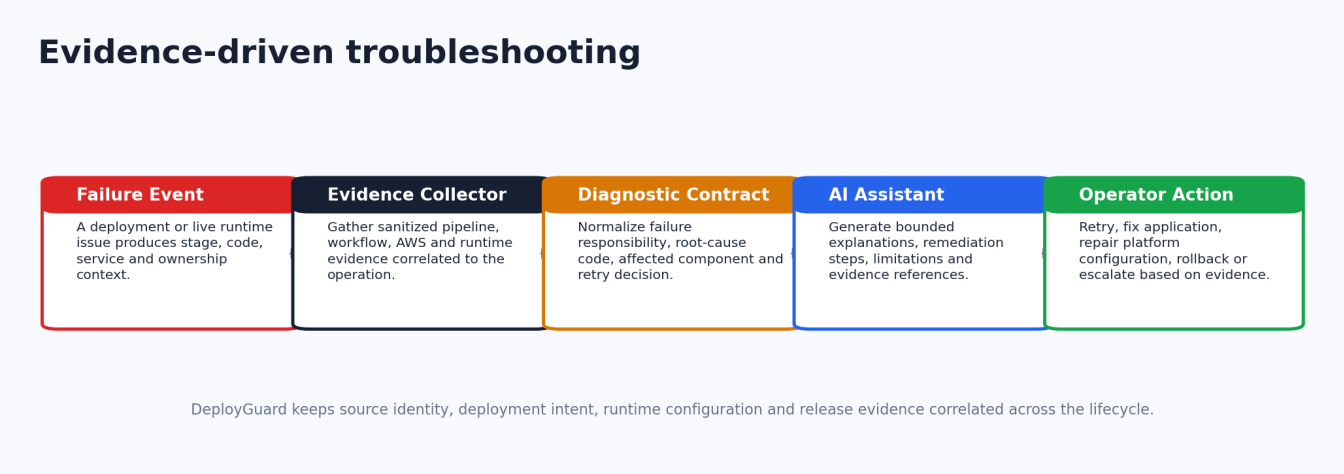


Figure 6. Evidence-first AI troubleshooting loop.

The troubleshooting subsystem is intentionally evidence-first. A failed deployment must contain sanitized persisted evidence before AI analysis becomes eligible. Live runtime troubleshooting similarly requires current generation-correlated runtime evidence. This prevents the AI layer from becoming an unaudited source of truth.

The evidence collector groups pipeline, cloud and runtime facts and removes unsafe or irrelevant content. The diagnostic layer preserves deterministic failure ownership and root-cause contracts. The AI provider receives bounded context and returns summary, root cause, technical details, remediation steps, evidence references, limitations, confidence and retry guidance.

If the external AI provider is unavailable, the platform can fall back to evidence-only analysis. This graceful degradation ensures that troubleshooting remains usable even when the model service is not available.

20. FinOps, Billing and Cost Governance

DeployGuard includes a FinOps module that generates project cost estimates from infrastructure plans, stores resource-level breakdowns, applies subscription-tier and warning-threshold policies, and can require explicit approval before continuing a deployment. Cost evidence is correlated with project and pipeline identities.

The product also includes billing accounts, subscriptions, invoices, payments, checkout sessions, usage counters and verified billing webhooks. Stripe integration provides a commercial billing path, while the deployment engine remains separated from billing-provider logic.

FinOps control	Purpose
Pre-deployment estimate	Make expected infrastructure cost visible before or during deployment.

Resource breakdown	Explain which resource classes contribute to monthly cost.
Warning threshold	Surface cost risk without necessarily blocking the operation.
Tier limit	Enforce subscription or product constraints.
Approval workflow	Pause a pipeline until an authorized user accepts the estimate.
Cost evidence	Persist cost outcome with pipeline/release context.

21. Frontend and Developer Experience

The React frontend exposes the deployment lifecycle as a product rather than as raw cloud console operations. Major surfaces include dashboard, projects, GitHub connection, new-project onboarding, project overview, pipeline execution, environment variables, infrastructure, metrics, troubleshooting, billing, audit logs and admin cloud cleanup.

The UI deliberately separates developer-facing concepts from provider-specific detail. A developer sees source, service, environment, build, deployment progress, health, release history, cost and recovery actions. Low-level cloud evidence remains available where it supports diagnosis, but it is not required for the normal deployment journey.

22. Testing and Certification Strategy

The repository contains an extensive verification harness across backend, frontend, infrastructure and full-stack behavior. The strategy is contract-oriented: tests verify source/build resolution, ports, configuration admission, runtime validation, lifecycle properties, release identity, failure ownership, database behavior, security controls, state safety, API equivalence, UI state projection and end-to-end interactions.

A critical FYP-II distinction is the difference between structural verification and live cloud proof. Static inspection or unit/contract tests can prove that an invariant is encoded, but a live deployment is required to claim end-to-end cloud success on a specific commit. The project therefore uses explicit evidence language rather than assuming that source correctness automatically guarantees an external cloud outcome.

Verification area	Examples of checks
Repository/build	service discovery, build-target resolution,

	application entrypoint, port resolution, multi-service boundaries
Configuration	environment ownership, secret isolation, runtime revisions, immutable fingerprints
CI/CD	workflow contract, dispatch state, external action pinning, release-result evidence
AWS runtime	ECS deployment, target health, direct redeploy, rollback, runtime validation
Infrastructure	Terraform plan/apply/destroy semantics, state lock, state recovery, lifecycle ownership
Security	vulnerability-scan controls, OIDC trust, failure sanitization, audit boundaries
Database	managed engine profile, persistent storage, grant/readiness reconciliation, schema integrity
UX	deploy journey, lifecycle presentation, recovery, monitoring, admin/readability, troubleshooting UI
Full stack	Playwright and real-stack certification harnesses

23. FYP-II Implementation Achievements

- Converted repository analysis into immutable, source-SHA-scoped deployment contracts.
- Implemented automatic service-port authority with conflict/invalid/unresolved failure modes.
- Built a complete source-to-container-to-AWS release workflow using GitHub Actions and Terraform.
- Integrated AWS OIDC role trust and lifecycle-specific capability verification.
- Implemented project-scoped ECS/Fargate runtime, ALB routing, CloudWatch logs and Secrets Manager integration.
- Added managed PostgreSQL, MySQL and MongoDB with encrypted persistent EFS storage and private service discovery.
- Added immutable runtime revisions, deployment generations, stable release history, redeploy and manual rollback.

- Implemented destroy verification, cloud inventory and cleanup/reconciliation services.
- Integrated container vulnerability scanning and structured failure diagnostics.
- Added metrics, monitoring, Prometheus/Grafana integration and developer observability views.
- Added evidence-driven AI troubleshooting with sanitization, provider fallback and auditability.
- Added FinOps cost estimation, approval policies, subscription/billing components and usage enforcement.
- Built a large contract and certification test surface for FYP-II validation.

24. Demonstration Plan

6. Open DeployGuard and authenticate.
7. Connect/select a known GitHub repository and branch.
8. Create a project and show detected deployable service and configuration.
9. Review environment variables / managed database choice where relevant.
10. Start deployment and show synchronized lifecycle stages.
11. Open the GitHub Actions run to show automated execution and AWS OIDC authentication.
12. Show the published image/release identity and AWS ECS/ALB evidence.
13. Open the LIVE application and verify public behavior.
14. Show monitoring/logs and stable release history.
15. Demonstrate rollback or redeploy evidence if time permits.
16. Execute Destroy and show resource lifecycle finalization.

25. Limitations

The current supported build contract is intentionally bounded to JavaScript and Python application classes. The platform is not yet a universal polyglot deployment system. Pure static repositories without an application server are outside the current contract.

The current AWS runtime is optimized for the FYP implementation and demonstration rather than for hyperscale multi-tenant economics. Per-project resources such as load balancers can become costly at very high deployment counts, and a mature commercial version would introduce shared routing, bin-packing and more aggressive resource pooling.

The system currently targets AWS as the primary runtime. The control plane is designed around explicit contracts, but true multi-cloud support would require provider-specific capability models, infrastructure modules, cost models and runtime verification adapters.

AI troubleshooting remains advisory. Deterministic platform evidence and lifecycle contracts remain authoritative, which is intentional for safety.

26. Future Work

- Shared ingress and resource pooling to reduce per-application infrastructure cost.
- Autoscaling policies based on CPU, memory, request rate and latency.

- Multi-region deployment and disaster-recovery policies.
- Additional build contracts for more ecosystems while preserving fail-closed admission.
- Provider adapters for lower-cost compute and multi-cloud deployment.
- Stronger supply-chain security: signed provenance, SBOM policies and image signature verification.
- Expanded SLOs, alerting and deployment-quality analytics.
- Database backup/restore automation exposed as a first-class user workflow.
- More advanced release strategies such as canary, weighted traffic and blue/green promotion.
- Scale testing against thousands of low-traffic applications and resource-pooling designs.

27. Conclusion

DeployGuard FYP-II demonstrates a complete implementation of a deployment control plane rather than a collection of isolated DevOps scripts. Its main engineering contribution is the deterministic chain from immutable source identity to canonical build/runtime contracts, governed cloud execution, verified release promotion and lifecycle history.

The implementation combines self-service platform engineering, Infrastructure as Code, containerized cloud runtime, embedded security, state reconciliation, observability, cost governance and evidence-driven troubleshooting. The result is a defensible FYP-II system whose architecture can be explained in terms of ownership, invariants and lifecycle correctness, while standard cloud tools remain supporting dependencies rather than the claimed innovation.

References

[1] Mateen Ali Anjum (2026). Platform engineering and internal developer portals: a multivocal literature review. Frontiers in Computer Science.

[2] Harshad Pitkar (2026). Platform Engineering and Developer Experience: A Systematic Review of Concepts, Benefits and Future Directions. World Journal of Advanced Engineering Technology and Sciences.

[3] Naveen Kumar Kasarla (2025). Implementing Infrastructure as Code (IaC) with Terraform for Scalable Cloud Deployments. Journal of Information Systems Engineering and Management.

[4] Manohar Marandi, A. Bertia, Salaja Silas (2023). Implementing and Automating Security Scanning to a DevSecOps CI/CD Pipeline. World Conference on Communication & Computing (WCONF).

[5] Youngsoo Kim, Cheolhee Park, Dowon Hong (2025). A Robust Framework for Comprehensive Container Image Vulnerability Assessment. IEEE Access 13, 35837-35847.

[6] Oluwatobiloba Odunsi, Aravind Mohan, S. A. Mostafa, Jianwu Wang (2025). CloudBot: Autonomous End-to-End Cloud Deployment from Code to Infrastructure. IEEE International Conference on Big Data.

[7] Claus Pahl, Ovgum Can Sezen, Florian Hofer (2026). Artificial Intelligence for Infrastructure-as-Code - A Systematic Literature Review. Electronics.

Appendix A - FYP-II Defense Positioning

Panel question	Recommended positioning
What is original in your project?	The deterministic deployment control plane: immutable source/build/runtime contracts, orchestration, security/cost admission, release generations, evidence, rollback/destroy correctness and troubleshooting UX.
Are AWS/Terraform/GitHub your project?	No. They are infrastructure dependencies. DeployGuard coordinates them through its own contracts, lifecycle model and developer-facing control plane.
Is this just CI/CD?	No. CI/CD executes work; DeployGuard owns repository interpretation, deployment admission, runtime configuration, infrastructure lifecycle, release truth, rollback/destroy, monitoring, cost and diagnostics.
Why is this FYP-II?	Because the focus is implemented system behavior, integration, lifecycle correctness, live deployment and verification - not only requirements or design.